LECTURE SERIES · CSE 2205 · DATABASE MANAGEMENT SYSTEM

Database Transactions

ACID, Concurrency Control & Recovery in PostgreSQL



Md. Rakib Trofder

Lecturer, Bangladesh Army University of Science & Technology

ACID · BEGIN/COMMIT · Schedules · Serializability · Isolation · Locking · Recovery

TODAY'S JOURNEY

Lecture Roadmap

01

Transactions & ACID

Definition, the bank-transfer example and the four ACID properties.

02

PostgreSQL in Practice

BEGIN, COMMIT, ROLLBACK, SAVEPOINT and error handling.

03

Schedules & Serializability

Conflicts, precedence graphs and view equivalence — with problems.

04

Recoverability

Recoverable, cascadeless and strict schedules.

05

Concurrency Control

Locks, 2PL, deadlocks, MVCC and isolation levels.

06

Logging & Recovery

Write-ahead logging, undo/redo and counting schedules.

THE UNIT OF WORK

What Is a Transaction?

A **transaction** is a sequence of database operations executed as **one logical unit of work** — either all of it happens, or none of it does.

T: Transfer 500 Tk from A to B

```
read(A)
A := A − 500
write(A)
read(B)
B := B + 500
write(B)
```

If it crashes midway…

After write(A) but before write(B), 500 Tk simply vanishes. The DBMS must undo the partial work.

Two basic operations

read(X) copies X from the database into a local buffer; **write(X)** copies it back.

FOUR GUARANTEES

The ACID Properties

Atomicity

All operations of a transaction are reflected, or none are.

`A − 500 and B + 500 both, or neither`

Consistency

A transaction run alone takes the database from one valid state to another.

`A + B is the same before and after`

Isolation

Concurrent transactions behave as if they ran one after another.

`Others never see A debited but B not credited`

Durability

Once committed, changes survive crashes and power failures.

`COMMIT returns → transfer is permanent`

UNDER THE HOOD

ACID in PostgreSQL

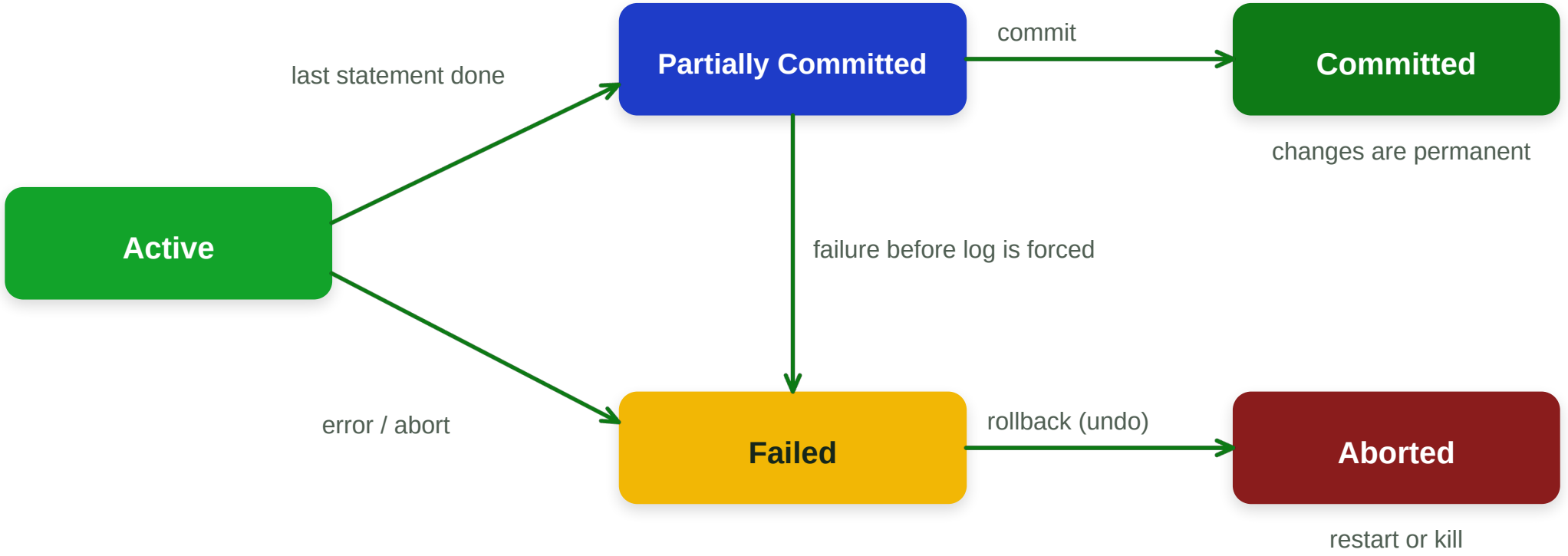
Each property is enforced by a specific DBMS component and a concrete PostgreSQL mechanism.

Property	Responsible component	PostgreSQL mechanism
Atomicity	Recovery manager	Write-Ahead Log (WAL) + transaction status; ROLLBACK discards the work
Consistency	Programmer + integrity checker	PRIMARY KEY, FOREIGN KEY, CHECK, NOT NULL, triggers
Isolation	Concurrency-control manager	MVCC snapshots, row-level locks, Serializable Snapshot Isolation (SSI)
Durability	Recovery manager	WAL flushed to disk at COMMIT, checkpoints, crash recovery by replaying WAL

LIFE CYCLE

Transaction States

Every transaction moves through these states between its first statement and its end.



POSTGRESQL

Transaction Control Commands

```
-- explicit transaction block
BEGIN;              -- or START TRANSACTION
  UPDATE account SET balance = balance - 500
   WHERE acc_no = 'A-101';
  UPDATE account SET balance = balance + 500
   WHERE acc_no = 'B-202';
COMMIT;             -- or END

-- undo everything since BEGIN
BEGIN;
  DELETE FROM account;
ROLLBACK;           -- or ABORT: nothing deleted
```

Autocommit

By default every single SQL statement is its own transaction.

BEGIN … COMMIT

Groups many statements into one atomic unit.

ROLLBACK

Discards all changes made since BEGIN.

WORKED EXAMPLE

Bank Transfer in PostgreSQL

1 · SCHEMA

```
CREATE TABLE account (
  acc_no  VARCHAR(10) PRIMARY KEY,
  owner   VARCHAR(50) NOT NULL,
  balance NUMERIC(12,2)
          CHECK (balance >= 0)
);
INSERT INTO account VALUES
  ('A-101', 'Rahim', 1000),
  ('B-202', 'Karim', 2000);
```

2 · TRANSFER 1500 TK

```
BEGIN;
UPDATE account
   SET balance = balance + 1500
 WHERE acc_no = 'B-202';   -- OK
UPDATE account
   SET balance = balance - 1500
 WHERE acc_no = 'A-101';
-- ERROR: new row for relation "account"
--   violates check constraint
--   "account_balance_check"
ROLLBACK;
```

Result: the CHECK constraint fails, so the whole transaction is rolled back — B's credit of 1500 Tk is undone too. Balances stay **A = 1000, B = 2000**. Atomicity protects consistency.

PARTIAL ROLLBACK

Savepoints

A **SAVEPOINT** marks a point inside a transaction that you can roll back to without abandoning the whole transaction.

```
BEGIN;
INSERT INTO orders VALUES (1, 'Laptop', 85000);
SAVEPOINT sp1;
INSERT INTO orders VALUES (2, 'Mouse', -500);
-- wrong price! undo only this part
ROLLBACK TO SAVEPOINT sp1;
INSERT INTO orders VALUES (2, 'Mouse', 500);
RELEASE SAVEPOINT sp1;
COMMIT;
```

Final contents of orders

id	item	price
1	Laptop	85000
2	Mouse	500

Remember

ROLLBACK TO keeps the savepoint; RELEASE removes it. Plain ROLLBACK still undoes everything.

POSTGRESQL BEHAVIOUR

Errors Inside a Transaction

In PostgreSQL, one failed statement puts the whole block into the **aborted** state. Every later command is refused until the block ends.

```
BEGIN;
INSERT INTO account VALUES ('C-303', 'Jamal', 500);
SELECT 1/0;
-- ERROR:  division by zero
INSERT INTO account VALUES ('D-404', 'Nila', 800);
-- ERROR:  current transaction is aborted,
--   commands ignored until end of transaction block
COMMIT;
-- ROLLBACK   (nothing is saved)
```

Why?

So that a half-finished transaction can never be committed by mistake.

Fix

Use a SAVEPOINT before risky statements, or ROLLBACK and retry.

ORDERING OPERATIONS

Schedules: Serial vs Concurrent

A **schedule** is the chronological order in which the operations of several transactions are executed. It keeps each transaction's own order.

Serial

One transaction runs completely, then the next. Always correct, but slow.

```
R1(A) W1(A) R1(B)
W1(B) R2(A) W2(A)
```

Concurrent

Operations interleave. Faster, but safe only if equivalent to some serial order.

```
R1(A) W1(A) R2(A)
W2(A) R1(B) W1(B)
```

Why concurrency?

Better CPU and disk utilisation, higher throughput and shorter waiting time for short transactions.

WHAT CAN GO WRONG

Concurrency Anomalies

Lost Update

Two transactions read-modify-write the same item; one overwrite erases the other.

```
R1(A) R2(A) W1(A) W2(A)
```

Dirty Read

T2 reads a value written by T1, then T1 rolls back.

```
W1(A) R2(A) A1
```

Non-repeatable Read

T1 reads the same item twice and gets two different values.

```
R1(A) W2(A) C2 R1(A)
```

Phantom Read

Re-running a query returns new rows inserted by another transaction.

```
COUNT(*) WHERE dept = 'CSE'
```

WORKED EXAMPLE

The Lost Update Problem

Initially **A = 100**. T1 withdraws 30, T2 deposits 50. The correct final value is **120**.

Time	T1	T2	A on disk
t1	R(A) → 100		100
t2		R(A) → 100	100
t3	A := A − 30 → 70		100
t4		A := A + 50 → 150	100
t5	W(A)		70
t6		W(A)	150

FINAL VALUE

150

instead of 120 — T1's withdrawal of 30 is lost.

Cure

Lock A for the whole read-modify-write, or use UPDATE … SET A = A − 30 in one statement.

TESTING CORRECTNESS

Conflict Serializability

Two operations **conflict** if they belong to **different** transactions, access the **same** item, and **at least one is a write**.

Pair (i ≠ j)	Conflict?	Edge
Ri(X), Rj(X)	No	—
Ri(X), Wj(X)	Yes (RW)	Ti → Tj
Wi(X), Rj(X)	Yes (WR)	Ti → Tj
Wi(X), Wj(X)	Yes (WW)	Ti → Tj

Precedence-graph test

1. Draw one node per transaction.
2. For each conflicting pair where Ti's operation comes first, add edge Ti → Tj.
3. If the graph has **no cycle**, the schedule is conflict serializable.
4. Any topological order of the graph is an equivalent serial schedule.

MATHEMATICAL PROBLEM 1

Is This Schedule Serializable?

S: R1(X), R2(Y), W1(X), R3(X), W2(Y), R3(Y), W3(X)

Step	T1	T2	T3
1	R(X)		
2		R(Y)	
3	W(X)		
4			R(X)
5		W(Y)	
6			R(Y)
7			W(X)

Tasks

(a) List all conflicting pairs.
(b) Draw the precedence graph.
(c) Is S conflict serializable? If yes, give every equivalent serial order.

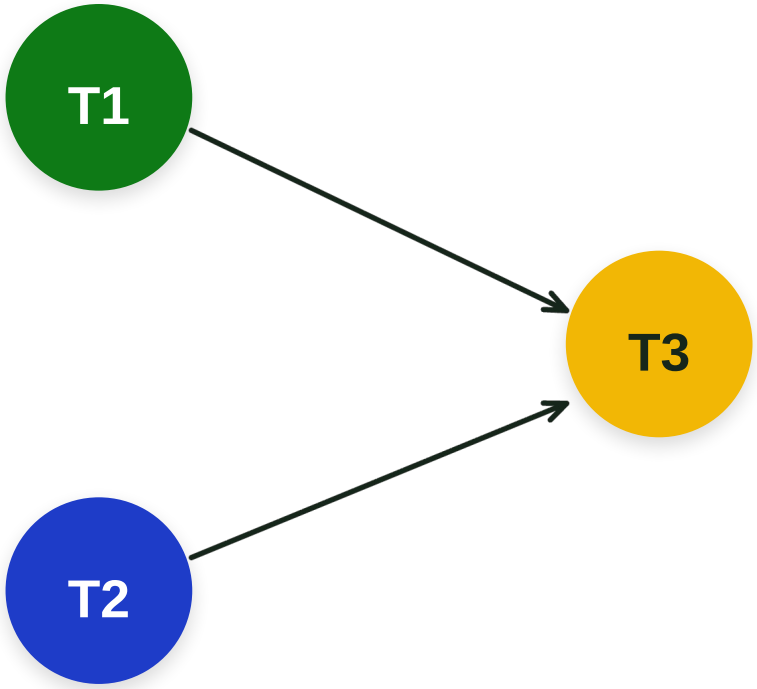
PROBLEM 1 · SOLUTION

Precedence Graph

Conflicting pairs

First	Then	Edge
R1(X)	W3(X)	T1 → T3
W1(X)	R3(X)	T1 → T3
W1(X)	W3(X)	T1 → T3
W2(Y)	R3(Y)	T2 → T3

No cycle → S is conflict serializable.
Serial orders: **T1, T2, T3** or **T2, T1, T3**.



Edges point into T3: it must run last.

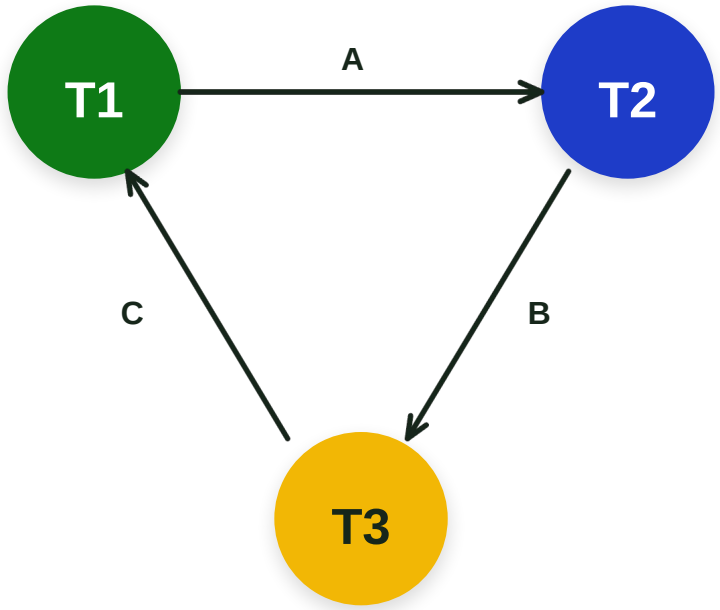
MATHEMATICAL PROBLEM 2

A Schedule With a Cycle

S: R1(A), W2(A), R2(B), W3(B), R3(C), W1(C)

Item	Conflict	Edge
A	R1(A) before W2(A)	T1 → T2
B	R2(B) before W3(B)	T2 → T3
C	R3(C) before W1(C)	T3 → T1

Cycle **T1 → T2 → T3 → T1** → S is **NOT** conflict serializable. No serial order can reproduce it.



MATHEMATICAL PROBLEM 2

A Schedule With a Cycle

S: R1(A), W2(A), R2(B), W3(B), R3(C), W1(C)

A WEAKER TEST · PROBLEM 3

View Serializability

S and S′ are view equivalent if, for every item X:

1. **Initial read** — the same transaction reads the initial value of X.
2. **Reads-from** — if Ti reads X written by Tj in S, it does so in S′ too.
3. **Final write** — the same transaction performs the last write of X.

Every conflict-serializable schedule is view serializable, not vice-versa. The extra cases always contain **blind writes**.

`S: R1(A), W2(A), W1(A), W3(A)`

Conflict test: R1(A)→W2(A) gives T1 → T2; W2(A)→W1(A) gives T2 → T1. Cycle → not conflict serializable.

View test vs. serial T1, T2, T3: T1 reads the initial A — same; no other reads — same; T3 writes A last — same.

→ **View serializable (W2, W3 are blind writes)**

PROBLEM 4

Recoverable Schedules

Recoverable

If Tj reads from Ti, then Ti commits before Tj commits.

Cascadeless

Transactions read only committed data, so one abort never forces others to abort.

Strict

No one reads or overwrites X until the last writer of X has committed or aborted.

Schedule	Classification	Reason
W1(A) R2(A) C2 C1	Not recoverable	T2 commits after reading T1's uncommitted A; if T1 aborts, T2 cannot be undone
W1(A) R2(A) C1 C2	Recoverable, not cascadeless	T1 commits first, but an abort of T1 would cascade to T2
W1(A) C1 R2(A) C2	Cascadeless and strict	T2 reads A only after T1 has committed

LOCK-BASED PROTOCOLS

Locks & Two-Phase Locking

Lock compatibility matrix

Requested	Held: S	Held: X
S (shared, read)	Granted	Wait
X (exclusive, write)	Wait	Wait

Many readers may share an item; a writer needs it alone.

Growing phase

A transaction may acquire locks but may not release any.

Shrinking phase

After its first release (the lock point is just before) it may not acquire any more.

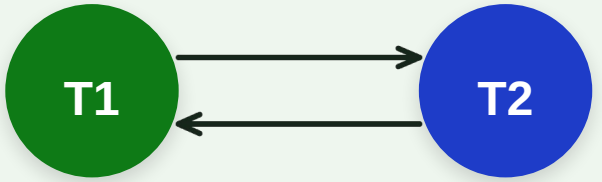
Strict 2PL

Hold all X locks until COMMIT/ABORT → cascadeless schedules.

WAITING FOREVER

Deadlocks in PostgreSQL

t	Session 1	Session 2
1	BEGIN; UPDATE … 'A-101'	
2		BEGIN; UPDATE … 'B-202'
3	UPDATE … 'B-202' → waits	
4		UPDATE … 'A-101' → waits
5	continues	ERROR: deadlock detected



Wait-for graph: a cycle = deadlock

PostgreSQL checks for cycles after **deadlock_timeout** (default 1 s) and aborts one transaction. **Prevent it:** update rows in a consistent order (e.g. by acc_no), keep transactions short, set **lock_timeout**.

MULTIVERSION CONCURRENCY CONTROL

MVCC in PostgreSQL

An UPDATE never overwrites a row. It writes a **new version** and marks the old one dead. Each transaction sees the versions valid in its **snapshot**.

After txn 745 runs UPDATE … balance = 950

xmin	xmax	acc_no	balance	Status
740	745	A-101	1000	dead (old)
745	0	A-101	950	live (new)

```
SELECT xmin, xmax, acc_no, balance
  FROM account;
SELECT pg_current_xact_id();  -- PG 13+
```

Readers never block writers

and writers never block readers.

xmin / xmax

Creator and deleter transaction IDs stored in every row version.

VACUUM

Reclaims dead versions no snapshot can see any more.

SQL STANDARD VS POSTGRESQL

Isolation Levels

Level	Dirty read	Non-repeatable read	Phantom read	Serialization anomaly
Read Uncommitted	Prevented in PG*	Possible	Possible	Possible
Read Committed (default)	Prevented	Possible	Possible	Possible
Repeatable Read	Prevented	Prevented	Prevented in PG*	Possible
Serializable	Prevented	Prevented	Prevented	Prevented

* The SQL standard allows these anomalies at that level, but PostgreSQL is stricter: Read Uncommitted behaves exactly like Read Committed, and Repeatable Read uses a single snapshot, so phantoms cannot appear.

POSTGRESQL

Choosing an Isolation Level

```
-- per transaction
BEGIN ISOLATION LEVEL REPEATABLE READ;
  SELECT SUM(balance) FROM account;
COMMIT;

BEGIN;
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
SHOW transaction_isolation;   -- serializable
COMMIT;

-- default for a whole database
ALTER DATABASE bankdb
  SET default_transaction_isolation
  = 'serializable';
```

Serialization failure

Under Repeatable Read or Serializable, a conflicting transaction can fail with SQLSTATE **40001**: could not serialize access.

Your job

Catch error 40001 in the application and simply **retry** the whole transaction.

LIVE DEMO · TWO PSQL SESSIONS

Read Committed vs Repeatable Read

t	Session A (reader)	Session B (writer)
1	BEGIN;	
2	SELECT balance FROM account WHERE acc_no='A-101'; → 1000	
3		UPDATE account SET balance=700 WHERE acc_no='A-101'; COMMIT;
4	same SELECT again → ?	
5	COMMIT;	

READ COMMITTED

Step 4 returns **700** — a non-repeatable read, because each statement takes a fresh snapshot.

REPEATABLE READ

Step 4 still returns **1000** — the snapshot was fixed at the first query in step 2.

EXPLICIT LOCKING

SELECT ... FOR UPDATE

```
-- Session A: book seat 12A safely
BEGIN;
SELECT * FROM seat
 WHERE seat_no = '12A' AND status = 'free'
   FOR UPDATE;           -- row is now locked
UPDATE seat SET status = 'booked',
                passenger = 'Nabila'
 WHERE seat_no = '12A';
COMMIT;
-- Session B, same SELECT ... FOR UPDATE:
-- waits for A, then finds 0 rows
```

FOR UPDATE

Exclusive lock on the selected rows.

FOR SHARE

Shared lock: others cannot update them.

NOWAIT

Error at once instead of waiting.

SKIP LOCKED

Skip locked rows: ideal for job queues.

LOGGING & RECOVERY

Write-Ahead Logging

Log record	Meaning
<Ti start>	Ti has begun
<Ti, X, V1, V2>	Ti changed X from old V1 to new V2
<Ti commit>	Ti committed
<Ti abort>	Ti rolled back

WAL rule: a log record must reach stable storage **before** the data page it describes. A transaction is committed once its <commit> record is on disk.

REDO

Ti has <start> and <commit> → set every X to its new value V2.

UNDO

Ti has <start> but no <commit>/<abort> → restore old values V1, newest first.

In PostgreSQL

WAL files in pg_wal; COMMIT waits for the WAL flush; checkpoints bound recovery time.

MATHEMATICAL PROBLEM 5

Recovery From the Log

LOG ON DISK AT CRASH

```
<T1 start>
<T1, A, 1000, 950>
<T2 start>
<T2, C, 700, 600>
<T1, B, 2000, 2050>
<T1 commit>
<T2, A, 950, 900>
— system crash —
```

Question: Which transactions are redone and undone? What are the final values of A, B and C?

Redo list

{ T1 } — has commit

Undo list

{ T2 } — no commit

Item	Undo T2	Redo T1	Final
A	900 → 950	950	950
B	—	2050	2050
C	600 → 700	—	700

MATHEMATICAL PROBLEM 6

How Many Schedules Exist?

For k transactions with n1, n2, …, nk operations, keeping each transaction's internal order:

$$\frac{(n1 + n2 + \ldots + nk)!}{n1! \cdot n2! \cdot \ldots \cdot nk!}$$

Only **k!** of them are serial.

Example (a)

T1 has 3 operations, T2 has 2.

```
5! / (3! · 2!) = 120 / 12 = 10
serial: 2! = 2
```

Example (b)

Three transactions, 2 operations each.

```
6! / (2!·2!·2!) = 720 / 8 = 90
serial: 3! = 6
```

So 84 of the 90 schedules are non-serial: we need serializability tests, not enumeration.

KEY TAKEAWAYS

Summary

ACID

A transaction is all-or-nothing, consistent, isolated and durable.

PostgreSQL control

BEGIN, COMMIT, ROLLBACK, SAVEPOINT; an error aborts the whole block.

Serializability

No cycle in the precedence graph → conflict serializable.

Recoverability

Commit order must follow reads-from; strict schedules avoid cascades.

Isolation

PostgreSQL uses MVCC; default is Read Committed; retry on error 40001.

Recovery

WAL first; redo committed, undo uncommitted transactions.

TRY IT YOURSELF

Practice Problems

1 Draw the precedence graph of **R1(A) R2(A) W1(A) R3(B) W2(B) W3(A)**. Is it conflict serializable? Give the serial order if any.

2 Classify **W1(X) R2(X) W2(Y) C2 A1** as recoverable, cascadeless or strict, and explain.

3 Write a PostgreSQL transaction that moves 300 Tk from A-101 to B-202, using a SAVEPOINT before the second UPDATE.

4 How many schedules are possible for T1 with 2 operations, T2 with 3 and T3 with 1? How many are serial?

CSE 2205 · DATABASE MANAGEMENT SYSTEM

Questions?

Next lecture: Concurrency Control Protocols in Depth

Md. Rakib Trofder

Lecturer, Bangladesh Army University of Science & Technology

